GLOBAL EDITION

Visual C#

How to Program

SIXTH EDITION

Paul Deitel • Harvey Deitel



Pearson

Visual C#®



HOW TO PROGRAM

This page intentionally left blank



Visual C#®

HOW TO PROGRAM

SIXTH EDITION

GLOBAL EDITION

Paul Deitel

Harvey Deitel

Deitel & Associates, Inc.



Boston Columbus Hoboken Indianapolis New York San Francisco
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal
Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Vice President, Editorial Director: ***Marcia Horton***
Acquisitions Editor: ***Tracy Johnson***
Editorial Assistant: ***Kristy Alaura***
Acquisitions Editor, Global Editions: ***Sourabh Maheshwari***
VP of Marketing: ***Christy Lesko***
Field Marketing Manager: ***Demetrius Hall***
Marketing Assistant: ***Jon Bryant***
Director of Product Management: ***Erin Gregg***
Team Lead, Program and Project Management: ***Scott Disanno***
Program Manager: ***Carole Snyder***
Project Manager: ***Robert Engelhardt***
Project Editor, Global Editions: ***K.K. Neelakantan***
Manufacturing Buyer, Higher Ed | RR Donnelley: ***Maura Zaldivar-Garcia***
Senior Manufacturing Controller, Global Editions: ***Trudy Kimber***
Media Production Manager, Global Editions: ***Vikram Kumar***
Cover Design: ***Lumina Datamatics***
R&P Manager: ***Ben Ferrini***
Inventory Manager: ***Ann Lam***
Cover Photo Credit: © ***Pichugin Dmitry/Shutterstock.com***

Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appear on page 6.

The authors and publisher of this book have used their best efforts in preparing this book. These efforts include the development, research, and testing of the theories and programs to determine their effectiveness. The authors and publisher make no warranty of any kind, expressed or implied, with regard to these programs or to the documentation contained in this book. The authors and publisher shall not be liable in any event for incidental or consequential damages in connection with, or arising out of, the furnishing, performance, or use of these programs.

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:
www.pearsonglobaleditions.com

© Pearson Education Limited 2018

The rights of Paul Deitel and Harvey Deitel to be identified as the authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

Authorized adaptation from the United States edition, entitled Visual C# How to Program,6th Edition, ISBN 978-0-13-460154-0 by Paul Deitel and Harvey Deitel published by Pearson Education © 2017.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6-10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1

ISBN-10: 1-292-15346-6

ISBN-13: 978-1-292-15346-9

Typeset by GEX Publishing Services

Printed and bound in Malaysia

In memory of William Siebert, Professor Emeritus of Electrical Engineering and Computer Science at MIT:

Your use of visualization techniques in your Signals and Systems lectures inspired the way generations of engineers, computer scientists, educators and authors present their work.

Harvey and Paul Deitel

Trademarks

DEITEL and the double-thumbs-up bug are registered trademarks of Deitel and Associates, Inc.

Microsoft® Windows®, and Microsoft Visual C#® are registered trademarks of the Microsoft Corporation in the U.S.A. And other countries. This book is not sponsored or endorsed by or affiliated with the Microsoft Corporation.

UNIX is a registered trademark of The Open Group.

Throughout this book, trademarks are used. Rather than put a trademark symbol in every occurrence of a trademarked name, we state that we are using the names in an editorial fashion only and to the benefit of the trademark owner, with no intention of infringement of the trademark.



Contents

Preface 23

Before You Begin 37

1 Introduction to Computers, the Internet and Visual C# 41

1.1 Introduction 42
1.2 Computers and the Internet in Industry and Research 42
1.3 Hardware and Software 45
1.3.1 Moore's Law 45
1.3.2 Computer Organization 46
1.4 Data Hierarchy 47
1.5 Machine Languages, Assembly Languages and High-Level Languages 50
1.6 Object Technology 51
1.7 Internet and World Wide Web 53
1.8 C# 55
1.8.1 Object-Oriented Programming 56
1.8.2 Event-Driven Programming 56
1.8.3 Visual Programming 56
1.8.4 Generic and Functional Programming 56
1.8.5 An International Standard 57
1.8.6 C# on Non-Windows Platforms 57
1.8.7 Internet and Web Programming 57
1.8.8 Asynchronous Programming with `async` and `await` 57
1.8.9 Other Key Programming Languages 58
1.9 Microsoft's .NET 60
1.9.1 .NET Framework 60
1.9.2 Common Language Runtime 60
1.9.3 Platform Independence 61
1.9.4 Language Interoperability 61
1.10 Microsoft's Windows® Operating System 61
1.11 Visual Studio Integrated Development Environment 63
1.12 **Painter** Test-Drive in Visual Studio Community 63

2 Introduction to Visual Studio and Visual Programming 73

2.1 Introduction 74
2.2 Overview of the Visual Studio Community 2015 IDE 74
2.2.1 Introduction to Visual Studio Community 2015 74
2.2.2 Visual Studio Themes 75
2.2.3 Links on the Start Page 75
2.2.4 Creating a New Project 76
2.2.5 **New Project** Dialog and Project Templates 77
2.2.6 Forms and Controls 78
2.3 Menu Bar and Toolbar 79
2.4 Navigating the Visual Studio IDE 82
2.4.1 **Solution Explorer** 83
2.4.2 **Toolbox** 84
2.4.3 **Properties** Window 84
2.5 Help Menu and Context-Sensitive Help 86
2.6 Visual Programming: Creating a Simple App that Displays Text and an Image 87
2.7 Wrap-Up 96
2.8 Web Resources 97

3 Introduction to C# App Programming 105

3.1 Introduction 106
3.2 Simple App: Displaying a Line of Text 106
3.2.1 Comments 107
3.2.2 `using` Directive 108
3.2.3 Blank Lines and Whitespace 109
3.2.4 Class Declaration 109
3.2.5 `Main` Method 111
3.2.6 Displaying a Line of Text 111
3.2.7 Matching Left ({) and Right (}) Braces 112
3.3 Creating a Simple App in Visual Studio 112
3.3.1 Creating the Console App 112
3.3.2 Changing the Name of the App File 114
3.3.3 Writing Code and Using *IntelliSense* 114
3.3.4 Compiling and Running the App 116
3.3.5 Errors, Error Messages and the **Error List** Window 117
3.4 Modifying Your Simple C# App 117
3.4.1 Displaying a Single Line of Text with Multiple Statements 118
3.4.2 Displaying Multiple Lines of Text with a Single Statement 118
3.5 String Interpolation 120
3.6 Another C# App: Adding Integers 121
3.6.1 Declaring the `int` Variable `number1` 122
3.6.2 Declaring Variables `number2` and `sum` 123

3.6.3 Prompting the User for Input 123
3.6.4 Reading a Value into Variable `number1` 123
3.6.5 Prompting the User for Input and Reading a Value into `number2` 124
3.6.6 Summing `number1` and `number2` 124
3.6.7 Displaying the `sum` with `string` Interpolation 125
3.6.8 Performing Calculations in Output Statements 125
3.7 Memory Concepts 125
3.8 Arithmetic 126
3.8.1 Arithmetic Expressions in Straight-Line Form 127
3.8.2 Parentheses for Grouping Subexpressions 127
3.8.3 Rules of Operator Precedence 127
3.8.4 Sample Algebraic and C# Expressions 128
3.8.5 Redundant Parentheses 129
3.9 Decision Making: Equality and Relational Operators 129
3.10 Wrap-Up 134

4 Introduction to Classes, Objects, Methods and `strings` 146

4.1 Introduction 147
4.2 Test-Driving an `Account` Class 148
4.2.1 Instantiating an Object—Keyword `new` 148
4.2.2 Calling Class `Account`'s `GetName` Method 149
4.2.3 Inputting a Name from the User 149
4.2.4 Calling Class `Account`'s `SetName` Method 150
4.3 `Account` Class with an Instance Variable and *Set* and *Get* Methods 150
4.3.1 `Account` Class Declaration 150
4.3.2 Keyword `class` and the Class Body 151
4.3.3 Instance Variable `name` of Type `string` 151
4.3.4 `SetName` Method 152
4.3.5 `GetName` Method 154
4.3.6 Access Modifiers `private` and `public` 154
4.3.7 `Account` UML Class Diagram 155
4.4 Creating, Compiling and Running a Visual C# Project with Two Classes 156
4.5 Software Engineering with *Set* and *Get* Methods 157
4.6 `Account` Class with a Property Rather Than *Set* and *Get* Methods 158
4.6.1 Class `AccountTest` Using Account's `Name` Property 158
4.6.2 `Account` Class with an Instance Variable and a Property 160
4.6.3 `Account` UML Class Diagram with a Property 162
4.7 Auto-Implemented Properties 162
4.8 `Account` Class: Initializing Objects with Constructors 163
4.8.1 Declaring an `Account` Constructor for Custom Object Initialization 163
4.8.2 Class `AccountTest`: Initializing `Account` Objects When They're Created 164

4.9 `Account` Class with a Balance; Processing Monetary Amounts 166
4.9.1 `Account` Class with a `decimal balance` Instance Variable 166
4.9.2 `AccountTest` Class That Uses `Account` Objects with Balances 169
4.10 Wrap-Up 173

5 Algorithm Development and Control Statements: Part 1 182

5.1 Introduction 183
5.2 Algorithms 184
5.3 Pseudocode 184
5.4 Control Structures 185
5.4.1 Sequence Structure 185
5.4.2 Selection Statements 186
5.4.3 Iteration Statements 187
5.4.4 Summary of Control Statements 187
5.5 `if` Single-Selection Statement 187
5.6 `if...else` Double-Selection Statement 188
5.6.1 Nested `if...else` Statements 189
5.6.2 Dangling-`else` Problem 191
5.6.3 Blocks 191
5.6.4 Conditional Operator (`?:`) 192
5.7 `Student` Class: Nested `if...else` Statements 193
5.8 `while` Iteration Statement 196
5.9 Formulating Algorithms: Counter-Controlled Iteration 197
5.9.1 Pseudocode Algorithm with Counter-Controlled Iteration 197
5.9.2 Implementing Counter-Controlled Iteration 198
5.9.3 Integer Division and Truncation 200
5.10 Formulating Algorithms: Sentinel-Controlled Iteration 201
5.10.1 Top-Down, Stepwise Refinement: The Top and First Refinement 201
5.10.2 Second Refinement 202
5.10.3 Implementing Sentinel-Controlled Iteration 204
5.10.4 Program Logic for Sentinel-Controlled Iteration 205
5.10.5 Braces in a `while` Statement 206
5.10.6 Converting Between Simple Types Explicitly and Implicitly 206
5.10.7 Formatting Floating-Point Numbers 207
5.11 Formulating Algorithms: Nested Control Statements 208
5.11.1 Problem Statement 208
5.11.2 Top-Down, Stepwise Refinement: Pseudocode Representation of the Top 209
5.11.3 Top-Down, Stepwise Refinement: First Refinement 209
5.11.4 Top-Down, Stepwise Refinement: Second Refinement 209
5.11.5 Complete Second Refinement of the Pseudocode 210
5.11.6 App That Implements the Pseudocode Algorithm 211
5.12 Compound Assignment Operators 213

5.13 Increment and Decrement Operators 213
5.13.1 Prefix Increment vs. Postfix Increment 214
5.13.2 Simplifying Increment Statements 215
5.13.3 Operator Precedence and Associativity 216
5.14 Simple Types 216
5.15 Wrap-Up 217

6 Control Statements: Part 2 232

6.1 Introduction 233
6.2 Essentials of Counter-Controlled Iteration 233
6.3 `for` Iteration Statement 235
6.3.1 A Closer Look at the `for` Statement's Header 236
6.3.2 General Format of a `for` Statement 236
6.3.3 Scope of a `for` Statement's Control Variable 236
6.3.4 Expressions in a `for` Statement's Header Are Optional 237
6.3.5 Placing Arithmetic Expressions in a `for` Statement's Header 237
6.3.6 Using a `for` Statement's Control Variable in the Statement's Body 238
6.3.7 UML Activity Diagram for the `for` Statement 238
6.4 Examples Using the `for` Statement 238
6.5 App: Summing Even Integers 239
6.6 App: Compound-Interest Calculations 240
6.6.1 Performing the Interest Calculations with Math Method `pow` 241
6.6.2 Formatting with Field Widths and Alignment 242
6.6.3 Caution: Do Not Use `float` or `double` for Monetary Amounts 243
6.7 `do...while` Iteration Statement 244
6.8 `switch` Multiple-Selection Statement 245
6.8.1 Using a `switch` Statement to Count A, B, C, D and F Grades 245
6.8.2 `switch` Statement UML Activity Diagram 249
6.8.3 Notes on the Expression in Each `case` of a `switch` 250
6.9 Class `AutoPolicy` Case Study: `strings` in `switch` Statements 251
6.10 `break` and `continue` Statements 253
6.10.1 `break` Statement 253
6.10.2 `continue` Statement 254
6.11 Logical Operators 255
6.11.1 Conditional AND (`&&`) Operator 256
6.11.2 Conditional OR (`||`) Operator 256
6.11.3 Short-Circuit Evaluation of Complex Conditions 257
6.11.4 Boolean Logical AND (`&`) and Boolean Logical OR (`|`) Operators 257
6.11.5 Boolean Logical Exclusive OR (`^`) 258
6.11.6 Logical Negation (`!`) Operator 258
6.11.7 Logical Operators Example 259
6.12 Structured-Programming Summary 261
6.13 Wrap-Up 266

7 Methods: A Deeper Look 277

7.1 Introduction 278
7.2 Packaging Code in C# 279
7.2.1 Modularizing Programs 279
7.2.2 Calling Methods 280
7.3 `static` Methods, `static` Variables and Class `Math` 280
7.3.1 `Math` Class Methods 281
7.3.2 `Math` Class Constants `PI` and `E` 282
7.3.3 Why Is `Main` Declared `static`? 282
7.3.4 Additional Comments About `Main` 283
7.4 Methods with Multiple Parameters 283
7.4.1 Keyword `static` 285
7.4.2 Method `Maximum` 285
7.4.3 Assembling `string`s with Concatenation 285
7.4.4 Breaking Apart Large `string` Literals 286
7.4.5 When to Declare Variables as Fields 287
7.4.6 Implementing Method `Maximum` by Reusing Method `Math.Max` 287
7.5 Notes on Using Methods 287
7.6 Argument Promotion and Casting 288
7.6.1 Promotion Rules 289
7.6.2 Sometimes Explicit Casts Are Required 289
7.7 The .NET Framework Class Library 290
7.8 Case Study: Random-Number Generation 292
7.8.1 Creating an Object of Type `Random` 292
7.8.2 Generating a Random Integer 292
7.8.3 Scaling the Random-Number Range 293
7.8.4 Shifting Random-Number Range 293
7.8.5 Combining Shifting and Scaling 293
7.8.6 Rolling a Six-Sided Die 293
7.8.7 Scaling and Shifting Random Numbers 296
7.8.8 Repeatability for Testing and Debugging 297
7.9 Case Study: A Game of Chance; Introducing Enumerations 297
7.9.1 Method `RollDice` 300
7.9.2 Method `Main`'s Local Variables 300
7.9.3 `enum` Type `Status` 301
7.9.4 The First Roll 301
7.9.5 `enum` Type `DiceNames` 301
7.9.6 Underlying Type of an `enum` 301
7.9.7 Comparing Integers and `enum` Constants 302
7.10 Scope of Declarations 302
7.11 Method-Call Stack and Activation Records 305
7.11.1 Method-Call Stack 305
7.11.2 Stack Frames 305
7.11.3 Local Variables and Stack Frames 306
7.11.4 Stack Overflow 306
7.11.5 Method-Call Stack in Action 306

7.12 Method Overloading 309
7.12.1 Declaring Overloaded Methods 309
7.12.2 Distinguishing Between Overloaded Methods 310
7.12.3 Return Types of Overloaded Methods 311
7.13 Optional Parameters 311
7.14 Named Parameters 312
7.15 C# 6 Expression-Bodied Methods and Properties 313
7.16 Recursion 314
7.16.1 Base Cases and Recursive Calls 314
7.16.2 Recursive Factorial Calculations 314
7.16.3 Implementing Factorial Recursively 315
7.17 Value Types vs. Reference Types 317
7.18 Passing Arguments By Value and By Reference 318
7.18.1 `ref` and `out` Parameters 319
7.18.2 Demonstrating `ref`, `out` and Value Parameters 320
7.19 Wrap-Up 322

8 Arrays; Introduction to Exception Handling 339

8.1 Introduction 340
8.2 Arrays 341
8.3 Declaring and Creating Arrays 342
8.4 Examples Using Arrays 343
8.4.1 Creating and Initializing an Array 343
8.4.2 Using an Array Initializer 344
8.4.3 Calculating a Value to Store in Each Array Element 345
8.4.4 Summing the Elements of an Array 347
8.4.5 Iterating Through Arrays with `foreach` 347
8.4.6 Using Bar Charts to Display Array Data Graphically; Introducing Type Inference with `var` 349
8.4.7 Using the Elements of an Array as Counters 352
8.5 Using Arrays to Analyze Survey Results; Intro to Exception Handling 353
8.5.1 Summarizing the Results 354
8.5.2 Exception Handling: Processing the Incorrect Response 355
8.5.3 The `try` Statement 355
8.5.4 Executing the `catch` Block 355
8.5.5 `Message` Property of the Exception Parameter 356
8.6 Case Study: Card Shuffling and Dealing Simulation 356
8.6.1 Class `Card` and Getter-Only Auto-Implemented Properties 356
8.6.2 Class `DeckOfCards` 357
8.6.3 Shuffling and Dealing Cards 360
8.7 Passing Arrays and Array Elements to Methods 361
8.8 Case Study: `GradeBook` Using an Array to Store Grades 363
8.9 Multidimensional Arrays 369
8.9.1 Rectangular Arrays 369

8.9.2 Jagged Arrays 370
8.9.3 Two-Dimensional Array Example: Displaying Element Values 371
8.10 Case Study: `GradeBook` Using a Rectangular Array 374
8.11 Variable-Length Argument Lists 380
8.12 Using Command-Line Arguments 382
8.13 (Optional) Passing Arrays by Value and by Reference 384
8.14 Wrap-Up 388

9 Introduction to LINQ and the `List` Collection 410

9.1 Introduction 411
9.2 Querying an Array of `int` Values Using LINQ 412
9.2.1 The `from` Clause 414
9.2.2 The `where` Clause 415
9.2.3 The `select` Clause 415
9.2.4 Iterating Through the Results of the LINQ Query 415
9.2.5 The `orderby` Clause 415
9.2.6 Interface `IEnumerable<T>` 416
9.3 Querying an Array of `Employee` Objects Using LINQ 416
9.3.1 Accessing the Properties of a LINQ Query's Range Variable 420
9.3.2 Sorting a LINQ Query's Results by Multiple Properties 420
9.3.3 `Any`, `First` and `Count` Extension Methods 420
9.3.4 Selecting a Property of an Object 420
9.3.5 Creating New Types in the `select` Clause of a LINQ Query 420
9.4 Introduction to Collections 421
9.4.1 `List<T>` Collection 421
9.4.2 Dynamically Resizing a `List<T>` Collection 422
9.5 Querying the Generic `List` Collection Using LINQ 426
9.5.1 The `let` Clause 428
9.5.2 Deferred Execution 428
9.5.3 Extension Methods `ToArray` and `ToList` 428
9.5.4 Collection Initializers 428
9.6 Wrap-Up 429
9.7 Deitel LINQ Resource Center 429

10 Classes and Objects: A Deeper Look 434

10.1 Introduction 435
10.2 `Time` Class Case Study; Throwing Exceptions 435
10.2.1 `Time1` Class Declaration 436
10.2.2 Using Class `Time1` 437
10.3 Controlling Access to Members 439
10.4 Referring to the Current Object's Members with the `this` Reference 440
10.5 `Time` Class Case Study: Overloaded Constructors 442
10.5.1 Class `Time2` with Overloaded Constructors 442
10.5.2 Using Class `Time2`'s Overloaded Constructors 446

10.6 Default and Parameterless Constructors 448
10.7 Composition 449
10.7.1 Class `Date` 449
10.7.2 Class `Employee` 451
10.7.3 Class `EmployeeTest` 452
10.8 Garbage Collection and Destructors 453
10.9 `static` Class Members 453
10.10 `readonly` Instance Variables 457
10.11 **Class View** and **Object Browser** 458
10.11.1 Using the **Class View** Window 458
10.11.2 Using the **Object Browser** 459
10.12 Object Initializers 460
10.13 Operator Overloading; Introducing `struct` 460
10.13.1 Creating Value Types with `struct` 461
10.13.2 Value Type `ComplexNumber` 461
10.13.3 Class `ComplexTest` 463
10.14 `Time` Class Case Study: Extension Methods 464
10.15 Wrap-Up 467

11 Object-Oriented Programming: Inheritance 475

11.1 Introduction 476
11.2 Base Classes and Derived Classes 477
11.3 `protected` Members 479
11.4 Relationship between Base Classes and Derived Classes 480
11.4.1 Creating and Using a `CommissionEmployee` Class 481
11.4.2 Creating a `BasePlusCommissionEmployee` Class without Using Inheritance 485
11.4.3 Creating a `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy 490
11.4.4 `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy Using `protected` Instance Variables 493
11.4.5 `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy Using `private` Instance Variables 496
11.5 Constructors in Derived Classes 500
11.6 Software Engineering with Inheritance 500
11.7 Class `object` 501
11.8 Wrap-Up 502

12 OOP: Polymorphism and Interfaces 508

12.1 Introduction 509
12.2 Polymorphism Examples 511
12.3 Demonstrating Polymorphic Behavior 512
12.4 Abstract Classes and Methods 515
12.5 Case Study: Payroll System Using Polymorphism 517
12.5.1 Creating Abstract Base Class `Employee` 518

12.5.2 Creating Concrete Derived Class `SalariedEmployee` 520
12.5.3 Creating Concrete Derived Class `HourlyEmployee` 522
12.5.4 Creating Concrete Derived Class `CommissionEmployee` 523
12.5.5 Creating Indirect Concrete Derived Class `BasePlusCommissionEmployee` 525
12.5.6 Polymorphic Processing, Operator `is` and Downcasting 526
12.5.7 Summary of the Allowed Assignments Between Base-Class and Derived-Class Variables 531
12.6 `sealed` Methods and Classes 532
12.7 Case Study: Creating and Using Interfaces 533
12.7.1 Developing an `IPayable` Hierarchy 534
12.7.2 Declaring Interface `IPayable` 536
12.7.3 Creating Class `Invoice` 536
12.7.4 Modifying Class `Employee` to Implement Interface `IPayable` 538
12.7.5 Using Interface `IPayable` to Process `Invoices` and `Employees` Polymorphically 539
12.7.6 Common Interfaces of the .NET Framework Class Library 541
12.8 Wrap-Up 542

13 Exception Handling: A Deeper Look 547

13.1 Introduction 548
13.2 Example: Divide by Zero without Exception Handling 549
13.2.1 Dividing By Zero 550
13.2.2 Enter a Non-Numeric Denominator 551
13.2.3 Unhandled Exceptions Terminate the App 552
13.3 Example: Handling `DivideByZeroExceptions` and `FormatExceptions` 552
13.3.1 Enclosing Code in a `try` Block 554
13.3.2 Catching Exceptions 554
13.3.3 Uncaught Exceptions 555
13.3.4 Termination Model of Exception Handling 556
13.3.5 Flow of Control When Exceptions Occur 556
13.4 .NET `Exception` Hierarchy 557
13.4.1 Class `SystemException` 557
13.4.2 Which Exceptions Might a Method Throw? 558
13.5 `finally` Block 559
13.5.1 Moving Resource-Release Code to a `finally` Block 559
13.5.2 Demonstrating the `finally` Block 560
13.5.3 Throwing Exceptions Using the `throw` Statement 564
13.5.4 Rethrowing Exceptions 564
13.5.5 Returning After a `finally` Block 565
13.6 The `using` Statement 566
13.7 `Exception` Properties 567
13.7.1 Property `InnerException` 567
13.7.2 Other `Exception` Properties 568
13.7.3 Demonstrating `Exception` Properties and Stack Unwinding 568

13.7.4 Throwing an `Exception` with an `InnerException` 570
13.7.5 Displaying Information About the `Exception` 571
13.8 User-Defined Exception Classes 571
13.9 Checking for `null` References; Introducing C# 6's `?.` Operator 575
13.9.1 Null-Conditional Operator (`?.`) 575
13.9.2 Revisiting Operators `is` and `as` 576
13.9.3 Nullable Types 576
13.9.4 Null Coalescing Operator (`??`) 577
13.10 Exception Filters and the C# 6 `when` Clause 577
13.11 Wrap-Up 578

14 Graphical User Interfaces with Windows Forms: Part 1 584

14.1 Introduction 585
14.2 Windows Forms 586
14.3 Event Handling 588
14.3.1 A Simple Event-Driven GUI 589
14.3.2 Auto-Generated GUI Code 591
14.3.3 Delegates and the Event-Handling Mechanism 593
14.3.4 Another Way to Create Event Handlers 594
14.3.5 Locating Event Information 595
14.4 Control Properties and Layout 596
14.4.1 Anchoring and Docking 597
14.4.2 Using Visual Studio To Edit a GUI's Layout 599
14.5 `Labels`, `TextBoxes` and `Buttons` 600
14.6 `GroupBoxes` and `Panels` 603
14.7 `CheckBoxes` and `RadioButtons` 606
14.7.1 `CheckBoxes` 606
14.7.2 Combining Font Styles with Bitwise Operators 608
14.7.3 `RadioButtons` 609
14.8 `PictureBoxes` 614
14.9 `ToolTips` 617
14.10 `NumericUpDown` Control 618
14.11 Mouse-Event Handling 621
14.12 Keyboard-Event Handling 623
14.13 Wrap-Up 627

15 Graphical User Interfaces with Windows Forms: Part 2 637

15.1 Introduction 638
15.2 Menus 638
15.3 `MonthCalendar` Control 648
15.4 `DateTimePicker` Control 649

15.5 LinkLabel Control 652
15.6 ListBox Control 655
15.7 CheckedListBox Control 660
15.8 ComboBox Control 663
15.9 TreeView Control 667
15.10 ListView Control 673
15.11 TabControl Control 679
15.12 Multiple Document Interface (MDI) Windows 683
15.13 Visual Inheritance 691
15.14 User-Defined Controls 696
15.15 Wrap-Up 699

16 Strings and Characters: A Deeper Look 707

16.1 Introduction 708
16.2 Fundamentals of Characters and Strings 709
16.3 string Constructors 710
16.4 string Indexer, Length Property and CopyTo Method 711
16.5 Comparing strings 712
16.6 Locating Characters and Substrings in strings 716
16.7 Extracting Substrings from strings 719
16.8 Concatenating strings 720
16.9 Miscellaneous string Methods 720
16.10 Class StringBuilder 722
16.11 Length and Capacity Properties, EnsureCapacity Method and Indexer of Class StringBuilder 723
16.12 Append and AppendFormat Methods of Class StringBuilder 725
16.13 Insert, Remove and Replace Methods of Class StringBuilder 727
16.14 Char Methods 730
16.15 Introduction to Regular Expressions (Online) 732
16.16 Wrap-Up 732

17 Files and Streams 739

17.1 Introduction 740
17.2 Files and Streams 740
17.3 Creating a Sequential-Access Text File 741
17.4 Reading Data from a Sequential-Access Text File 750
17.5 Case Study: Credit-Inquiry Program 754
17.6 Serialization 759
17.7 Creating a Sequential-Access File Using Object Serialization 760
17.8 Reading and Deserializing Data from a Binary File 764
17.9 Classes File and Directory 767
17.9.1 Demonstrating Classes File and Directory 768
17.9.2 Searching Directories with LINQ 771
17.10 Wrap-Up 775

18 Searching and Sorting 782

18.1 Introduction 783
18.2 Searching Algorithms 784
18.2.1 Linear Search 784
18.2.2 Binary Search 788
18.3 Sorting Algorithms 792
18.3.1 Selection Sort 793
18.3.2 Insertion Sort 796
18.3.3 Merge Sort 800
18.4 Summary of the Efficiency of Searching and Sorting Algorithms 806
18.5 Wrap-Up 807

19 Custom Linked Data Structures 812

19.1 Introduction 813
19.2 Simple-Type `structs`, Boxing and Unboxing 813
19.3 Self-Referential Classes 814
19.4 Linked Lists 815
19.5 Stacks 828
19.6 Queues 832
19.7 Trees 835
19.7.1 Binary Search Tree of Integer Values 836
19.7.2 Binary Search Tree of `IComparable` Objects 843
19.8 Wrap-Up 849

20 Generics 855

20.1 Introduction 856
20.2 Motivation for Generic Methods 857
20.3 Generic-Method Implementation 859
20.4 Type Constraints 862
20.4.1 `IComparable<T>` Interface 862
20.4.2 Specifying Type Constraints 862
20.5 Overloading Generic Methods 865
20.6 Generic Classes 865
20.7 Wrap-Up 875

21 Generic Collections; Functional Programming with LINQ/PLINQ 881

21.1 Introduction 882
21.2 Collections Overview 884
21.3 Class `Array` and Enumerators 886
21.3.1 C# 6 `using static` Directive 888
21.3.2 Class `UsingArray`'s `static` Fields 889
21.3.3 `Array` Method `Sort` 889

21.3.4 `Array` Method `Copy` 889
21.3.5 `Array` Method `BinarySearch` 889
21.3.6 `Array` Method `GetEnumerator` and Interface `IEnumerator` 889
21.3.7 Iterating Over a Collection with `foreach` 890
21.3.8 `Array` Methods `Clear`, `IndexOf`, `LastIndexOf` and `Reverse` 890
21.4 `Dictionary` Collections 890
21.4.1 `Dictionary` Fundamentals 891
21.4.2 Using the `SortedDictionary` Collection 892
21.5 Generic `LinkedList` Collection 896
21.6 C# 6 Null Conditional Operator `?[]` 900
21.7 C# 6 `Dictionary` Initializers and Collection Initializers 901
21.8 Delegates 901
21.8.1 Declaring a Delegate Type 903
21.8.2 Declaring a Delegate Variable 903
21.8.3 Delegate Parameters 904
21.8.4 Passing a Method Name Directly to a Delegate Parameter 904
21.9 Lambda Expressions 904
21.9.1 Expression Lambdas 906
21.9.2 Assigning Lambdas to Delegate Variables 907
21.9.3 Explicitly Typed Lambda Parameters 907
21.9.4 Statement Lambdas 907
21.10 Introduction to Functional Programming 907
21.11 Functional Programming with LINQ Method-Call Syntax and Lambdas 909
21.11.1 LINQ Extension Methods `Min`, `Max`, `Sum` and `Average` 912
21.11.2 `Aggregate` Extension Method for Reduction Operations 912
21.11.3 The `Where` Extension Method for Filtering Operations 914
21.11.4 `Select` Extension Method for Mapping Operations 915
21.12 PLINQ: Improving LINQ to Objects Performance with Multicore 915
21.13 (Optional) Covariance and Contravariance for Generic Types 919
21.14 Wrap-Up 921

22 Databases and LINQ 933

22.1 Introduction 934
22.2 Relational Databases 935
22.3 A Books Database 936
22.4 LINQ to Entities and the ADO.NET Entity Framework 940
22.5 Querying a Database with LINQ 941
22.5.1 Creating the ADO.NET Entity Data Model Class Library 943
22.5.2 Creating a Windows Forms Project and Configuring It to Use the Entity Data Model 947
22.5.3 Data Bindings Between Controls and the Entity Data Model 949
22.6 Dynamically Binding Query Results 955
22.6.1 Creating the **Display Query Results** GUI 956
22.6.2 Coding the **Display Query Results** App 957
22.7 Retrieving Data from Multiple Tables with LINQ 959

22.8 Creating a Master/Detail View App 965
22.8.1 Creating the Master/Detail GUI 965
22.8.2 Coding the Master/Detail App 967
22.9 Address Book Case Study 968
22.9.1 Creating the **Address Book** App's GUI 970
22.9.2 Coding the **Address Book** App 971
22.10 Tools and Web Resources 975
22.11 Wrap-Up 975

23 Asynchronous Programming with `async` and `await` 982

23.1 Introduction 983
23.2 Basics of `async` and `await` 985
23.2.1 `async` Modifier 985
23.2.2 `await` Expression 985
23.2.3 `async`, `await` and Threads 985
23.3 Executing an Asynchronous Task from a GUI App 986
23.3.1 Performing a Task Asynchronously 986
23.3.2 Method `calculateButton_Click` 988
23.3.3 `Task` Method `Run`: Executing Asynchronously in a Separate Thread 989
23.3.4 `await`ing the Result 989
23.3.5 Calculating the Next Fibonacci Value Synchronously 989
23.4 Sequential Execution of Two Compute-Intensive Tasks 990
23.5 Asynchronous Execution of Two Compute-Intensive Tasks 992
23.5.1 `await`ing Multiple `Task`s with `Task` Method `WhenAll` 995
23.5.2 Method `StartFibonacci` 996
23.5.3 Modifying a GUI from a Separate Thread 996
23.5.4 `await`ing One of Several Tasks with `Task` Method `WhenAny` 996
23.6 Invoking a Flickr Web Service Asynchronously with `HttpClient` 997
23.6.1 Using Class `HttpClient` to Invoke a Web Service 1001
23.6.2 Invoking the Flickr Web Service's `flickr.photos.search` Method 1001
23.6.3 Processing the XML Response 1002
23.6.4 Binding the Photo Titles to the `ListBox` 1003
23.6.5 Asynchronously Downloading an Image's Bytes 1004
23.7 Displaying an Asynchronous Task's Progress 1004
23.8 Wrap-Up 1008

Chapters on the Web 1015

A Operator Precedence Chart 1016

B Simple Types 1018

C ASCII Character Set 1020

Appendices on the Web 1021

Index 1023

Online Topics

PDFs presenting additional topics for advanced college courses and professionals are available through the book's Companion Website:

```
http://www.pearsonglobaleditions.com/deitel
```

New copies of this book come with a Companion Website access code that's located on the book's inside front cover. If the access code is already visible or there is no card, you purchased a used book or an edition that does not come with an access code.

Web App Development with ASP.NET

XML and LINQ to XML

Universal Windows Platform (UWP) GUI, Graphics, Multimedia and XAML

REST Web Services

Cloud Computing with Microsoft Azure™

Windows Presentation Foundation (WPF) GUI, Graphics, Multimedia and XAML

ATM Case Study, Part 1: Object-Oriented Design with the UML

ATM Case Study, Part 2: Implementing an Object-Oriented Design in C#

Using the Visual Studio Debugger



Preface

Welcome to the world of desktop, mobile and web app development with Microsoft's® Visual C#® programming language. *Visual C# How to Program, 6/e* is based on C# 6[1] and related Microsoft software technologies. You'll be using the .NET platform and the Visual Studio® Integrated Development Environment on which you'll conveniently write, test and debug your applications and run them on Windows® devices. The Windows operating system runs on desktop and notebook computers, mobile phones and tablets, game systems and a great variety of devices associated with the emerging "Internet of Things."

We believe that this book and its supplements for students and instructors will give you an informative, engaging, challenging and entertaining introduction to Visual C#. The book presents leading-edge computing technologies in a friendly manner appropriate for introductory college course sequences, based on the curriculum recommendations of two key professional organizations—the ACM and the IEEE.[2]

You'll study four of today's most popular programming paradigms:

- object-oriented programming,
- structured programming,
- generic programming and
- functional programming (new in this edition).

At the heart of the book is the Deitel signature *live-code approach*—rather than using code snippets, we generally present concepts in the context of complete working programs followed by sample executions. We include a broad range of example programs and exercises selected from computer science, business, education, social issues, personal utilities, sports, mathematics, puzzles, simulation, game playing, graphics, multimedia and many other areas. We also provide abundant tables, line drawings and UML diagrams for a more visual learning experience.

1. At the time of this writing, Microsoft has not yet released the official C# 6 Specification. To view an unofficial copy, visit `https://github.com/ljw1004/csharpspec/blob/gh-pages/README.md`
2. These recommendations include *Computer Science Curricula 2013 Curriculum Guidelines for Undergraduate Degree Programs in Computer Science*, December 20, 2013, The Joint Task Force on Computing Curricula, Association for Computing Machinery (ACM), IEEE Computer Society.

Read the Before You Begin section after this Preface for instructions on setting up your computer to run the hundreds of code examples and to enable you to develop your own C# apps. The source code for all of the book's examples is available at

```
http://www.pearsonglobaleditions.com/deitel
```

Use the source code we provide to compile and run each program as you study it—this will help you master Visual C# and related Microsoft technologies faster and at a deeper level. Most of the book's examples work in Visual Studio on Windows 7, 8 or 10 (there is no 9). The code examples for the online presentation of the Universal Windows Platform (UWP) and XAML specifically require Windows 10.

Contacting the Authors

As you read the book, if you have a question, we're easy to reach at

```
deitel@deitel.com
```

We'll respond promptly.

Join the Deitel & Associates, Inc. Social Media Communities

Subscribe to the *Deitel® Buzz Online* newsletter

```
http://www.deitel.com/newsletter/subscribe.html
```

and join the Deitel social media communities on

- Facebook®—`http://facebook.com/DeitelFan`
- LinkedIn®—`http://linkedin.com/company/deitel-&-associates`
- YouTube®—`http://youtube.com/DeitelTV`
- Twitter®—`http://twitter.com/Deitel`
- Instagram®—`http://instagram.com/DeitelFan`
- Google+™—`http://google.com/+DeitelFan`

Object-Oriented Programming with an Early Objects Approach

The book introduces the basic concepts and terminology of object-oriented programming in Chapter 1. In Chapter 2, you'll *visually* manipulate objects, such as labels and images. In Chapter 3, Introduction to C# App Programming, you'll write Visual C# program code that manipulates preexisting objects. You'll develop your first customized classes and objects in Chapter 4. Presenting objects and classes early gets you "thinking about objects" immediately and mastering these concepts more thoroughly.

Our early objects presentation continues in Chapters 5–9 with a variety of straightforward case studies. In Chapters 10–12, we take a deeper look at classes and objects,

present inheritance, interfaces and polymorphism, then use those concepts throughout the remainder of the book.

New C# 6 Features

6 We introduce key new C# 6 language features throughout the book (Fig. 1)—each defining occurrence is marked with a "6" margin icon as shown next to this paragraph.

C# 6 new language feature	First introduced in
`string` interpolation	Section 3.5
expression bodied methods and `get` accessors	Section 7.15
auto-implemented property initializers	Section 8.6.1
getter-only auto-implemented properties	Section 8.6.1
`nameof` operator	Section 10.5.1
null-conditional operator (`?.`)	Section 13.9.1
`when` clause for exception filtering	Section 13.10
`using static` directive	Section 21.3.1
null conditional operator (`?[]`)	Section 21.6
collection initializers for any collection with an `Add` extension method	Section 21.7
index initializers	Section 21.7

Fig. 1 | C# 6 new language features.

Interesting, Entertaining and Challenging Exercises

The book contains hundreds of exercises to practice the skills you learn. Extensive self-review exercises and answers are included for self-study. Also, each chapter concludes with a substantial set of exercises, which generally include

- simple recall of important terminology and concepts,
- identifying the errors in code samples,
- writing individual program statements,
- writing methods to perform specific tasks,
- writing C# classes,
- writing complete programs and
- implementing major projects.

Figure 2 lists only a sample of the book's hundreds of exercises, including selections from our *Making-a-Difference* exercises set, which encourage you to use computers and the Internet to research and work on significant social problems. We hope you'll approach these exercises in the context of your own values, politics and beliefs. The solutions to most of the book's exercises are available only to college instructors who have adopted the book for their courses. See "Instructor Supplements" later in this Preface.

A sampling of the book's exercises

Carbon Footprint Calculator
Body-Mass-Index Calculator
Attributes of Hybrid Vehicles
Gender Neutrality
Stopwatch GUI
Login Form GUI
Calculator GUI
Alarm Clock GUI
Radio GUI
Displaying Shapes
Odd or Even?
Is a Number a Multiple of Another?
Separating an Integer's Digits
Multiplication Table of a Number
Account Class
Student Record Class
Asset Class
Coaching Class
Removing Duplicated Code
Target-Heart-Rate Calculator
Computerizing Health Records
Inventory Level Calculator
Sales-Commission Calculator
Discount Calculator
Find the Two Largest Numbers
Dangling-else Problem
Expanded Form
Decimal Equivalent of a Binary Number
Type of Parallelogram
Permutations
Infinite Series: Mathematical Constant *e*
World Population Growth
Enforcing Privacy with Cryptography
Bar Chart Display
Prime Numbers
Calculating Sales
Car-Pool Savings Calculator
Gas Mileage Calculator
Calculating the Value of log (*x*)
Pythagorean Triples
Global Warming Facts Quiz
Tax Plan Alternative: The "FairTax"
Rounding to a Specific Decimal Place
Hypotenuse of a Right Triangle
Displaying a Hollow Right-Isosceles Triangle of Any Character
Separating Digits
Temperature Conversions
Amicable Numbers
Prime Numbers
Reversing Digits
Letter Grades to a Four-Point Scale
Coin Tossing
Guess-the-Number Game
Distance Between Two Points
Craps Game with Betting
Towers of Hanoi
Computer-Assisted Instruction
Wages Rate
Identifying Multiples of Eight
Dice Game of Craps
Airline Reservations System
Knight's Tour Chess Puzzle
Eight Queens Chess Puzzle
Sieve of Eratosthenes
Tortoise and the Hare
Merging Arrays
Building Your Own Computer (Virtual Machine)
Polling
Querying an Array of Invoice Objects
Name Connector
Hemisphere Class
Depreciating-Value Class
Set of Integers
RationalNumber Class
HugeInteger Class
Tic-Tac-Toe
ComplexNumber Class
Vehicle Inheritance Hierarchy
Payroll System
Accounts Payable System
Polymorphic Banking Program
CarbonFootprint Interface: Polymorphism
Length Conversions
Painter
Guess the Number Game
Ecofont
Typing Tutor
Restaurant Bill Calculator
Story Writer
Pig Latin
Cooking with Healthier Ingredients
Spam Elimination
SMS Language
File of Product Details
Telephone-Number Words
Student Poll
Phishing Scanner
Bucket Sort
Palindromes
Evaluating Expressions with a Stack
Building Your Own Compiler
Generic Linear Search
SortedDictionary of Colors
Prime Factorization
Bucket Sort with LinkedList<int>
Sieve of Eratosthenes with BitArray
Credit-Inquiry Program
Rolling a Die 60,000,000 Times
Baseball Database App
Parsing with LINQ to XML
I/O-Bound vs. Compute-Bound Apps
Recursive Fibonacci

Fig. 2 | A sampling of the book's exercises.

A Tour of the Book

This section discusses the book's modular organization to help instructors plan their syllabi.

Introduction to Computing, Visual C# and Visual Studio 2015 Community Edition
The chapters in this module of the book

- Chapter 1, Introduction to Computers, the Internet and Visual C#
- Chapter 2, Introduction to Visual Studio and Visual Programming

introduce hardware and software fundamentals, Microsoft's .NET platform and Visual Programming. The vast majority of the book's examples will run on Windows 7, 8 and 10 using the *Visual Studio 2015 Community* edition with which we test-drive a fun **Painter** app in Section 1.12. Chapter 1's introduction to object-oriented programming defines key terminology and discusses important concepts on which the rest of the book depends.

Introduction to C# Fundamentals
The chapters in this module of the book

- Chapter 3, Introduction to C# App Programming
- Chapter 4, Introduction to Classes, Objects, Methods and Strings
- Chapter 5, Algorithm Development and Control Statements: Part 1
- Chapter 6, Control Statements: Part 2
- Chapter 7, Methods: A Deeper Look
- Chapter 8, Arrays; Introduction to Exception Handling

present rich coverage of C# programming fundamentals (data types, operators, control statements, methods and arrays) and introduce object-oriented programming through a series of case studies. These chapters should be covered in order. Chapters 5 and 6 present a friendly treatment of control statements and problem solving. Chapters 7 and 8 present rich treatments of methods and arrays, respectively. Chapter 8 briefly introduces exception handling with an example that demonstrates attempting to access an element outside an array's bounds.

Object-Oriented Programming: A Deeper Look
The chapters in this module of the book

- Chapter 9, Introduction to LINQ and the `List` Collection
- Chapter 10, Classes and Objects: A Deeper Look
- Chapter 11, Object-Oriented Programming: Inheritance
- Chapter 12, OOP: Polymorphism and Interfaces
- Chapter 13, Exception Handling: A Deeper Look

provide a deeper look at object-oriented programming, including classes, objects, inheritance, polymorphism, interfaces and exception handling. An optional online two-chapter case study on designing and implementing the object-oriented software for a simple ATM is described later in this preface.

Chapter 9 introduces Microsoft's Language Integrated Query (LINQ) technology, which provides a uniform syntax for manipulating data from various data sources, such as

arrays, collections and, as you'll see in later chapters, databases and XML. Chapter 9 is intentionally simple and brief to encourage instructors to begin covering LINQ technology early. Section 9.4 introduces the `List` collection, which we use in Chapter 12. Later in the book, we take a deeper look at LINQ, using LINQ to Entities (for querying databases) and LINQ to XML. Chapter 9's LINQ coverage can be deferred if you're in a course which either skips LINQ or defers coverage until later in the book—it's required for one example in Chapter 17 (Fig. 17.6) and many of the later chapters starting with Chapter 22, Databases and LINQ.

Windows `Forms` Graphical User Interfaces (GUIs)
The chapters in this module of the book

- Chapter 14, Graphical User Interfaces with Windows Forms: Part 1
- Chapter 15, Graphical User Interfaces with Windows Forms: Part 2

present a detailed introduction to building GUIs using Windows `Forms`—instructors teaching Visual C# still largely prefer Windows `Forms` for their classes. Many of the examples in GUI Chapters 14–15 can be presented after Chapter 4. We also use Windows `Forms` GUIs in several other print and online chapters.

There are two other GUI technologies in Windows—Windows Presentation Foundation (WPF) and Universal Windows Platform (UWP). We provide optional online treatments of both.[3]

Strings and Files
The chapters in this module of the book

- Chapter 16, Strings and Characters: A Deeper Look
- Chapter 17, Files and Streams

present `string` processing and file processing, respectively. We introduce `string`s beginning in Chapter 4 and use them throughout the book. Chapter 16 investigates `string`s in more detail. Most of Chapter 16's examples can be presented at any point after Chapter 4. Chapter 17 introduces text-file processing and object-serialization for inputting and outputting entire objects. Chapter 17 requires Windows `Forms` concepts presented in Chapter 14.

Searching, Sorting and Generic Data Structures
The chapters in this module of the book:

- Chapter 18, Searching and Sorting
- Chapter 19, Custom Linked Data Structures
- Chapter 20, Generics
- Chapter 21, Generic Collections; Functional Programming with LINQ/PLINQ

introduce searching, sorting and data structures. Most C# programmers should use .NET's *built-in* searching, sorting and generic collections (prepackaged data structures) capabilities, which are discussed in Chapter 21. For instructors who wish to present how to implement customized searching, sorting and data structures capabilities, we provide Chapters 18–20, which require the concepts presented in Chapters 3–8 and 10–13. Chapter 18 presents sev-

3. As of Summer 2016, Windows Forms, WPF and UWP apps all can be posted for distribution via the Windows Store. See `http://bit.ly/DesktopToUWP` for more information.

eral searching and sorting algorithms and uses Big O notation to help you compare how hard each algorithm works to do its job—the code examples use especially visual outputs to show how the algorithms work. In Chapter 19, we show how to implement your own custom data structures, including lists, stacks, queues and binary trees. The data structures in Chapter 19 store references to `objects`. Chapter 20 introduces C# generics and demonstrates how to create type-safe generic methods and a type-safe generic stack data structure.

Functional Programming with LINQ, PLINQ, Lambdas, Delegates and Immutability
In addition to generic collections, Chapter 21 now introduces functional programming, showing how to use it with LINQ to Objects to write code more concisely and with fewer bugs than programs written using previous techniques. In Section 21.12, with one additional method call, we'll demonstrate how PLINQ (Parallel LINQ) can improve LINQ to Objects performance substantially on multicore systems. The chapter's exercises also ask you to reimplement earlier examples using functional-programming techniques.

Database with LINQ to Entities and SQL Server
The chapter in this module

- Chapter 22, Databases and LINQ

presents a novice-friendly introduction to database programming with the ADO.NET Entity Framework, LINQ to Entities and Microsoft's free version of SQL Server that's installed with the Visual Studio 2015 Community edition. The chapter's examples require C#, object-oriented programming and Windows `Forms` concepts presented in Chapters 3–14. Several online chapters require the techniques presented in this chapter.

Asynchronous Programming
The chapter in this module

- Chapter 23, Asynchronous Programming with `async` and `await`

shows how to take advantage of multicore architectures by writing applications that can process tasks asynchronously, which can improve app performance and GUI responsiveness in apps with long-running or compute-intensive tasks. The `async` modifier and `await` operator greatly simplify asynchronous programming, reduce errors and enable your apps to take advantage of the processing power in today's multicore computers, smartphones and tablets. In this edition, we added a case study that uses the Task Parallel Library (TPL), `async` and `await` in a GUI app—we keep a progress bar moving along in the GUI thread in parallel with a lengthy, compute-intensive calculation in another thread.

A Tour of the Online Content

The printed book contains the core content (Chapters 1–23) for introductory and intermediate course sequences. Several optional online topics for advanced courses and professionals are available on the book's password-protected Companion Website

```
http://www.pearsonglobaleditions.com/deitel
```

New copies of this book come with a Companion Website access code that's located on the book's inside front cover. If the access code is already visible or there isn't an access code, you purchased a used book or an edition that does not come with an access code. Figure 3 lists the online topics, and Figure 4 lists a sample of the associated exercises.

Online topics
Web App Development with ASP.NET
XML and LINQ to XML
Universal Windows Platform (UWP) GUI, Graphics, Multimedia and XAML
REST Web Services
Cloud Computing with Microsoft Azure™
Using the Visual Studio Debugger
(Optional) Windows Presentation Foundation (WPF) GUI, Graphics, Multimedia and XAML
(Optional) ATM Case Study, Part 1: Object-Oriented Design with the UML
(Optional) ATM Case Study, Part 2: Implementing an OO Design in C#

Fig. 3 | Online topics on the *Visual C# How to Program, 6/e* Companion Website.

A sampling of the online chapters' exercises		
Guestbook App	College Loan Payoff Calculator App	Favorite Flickr Searches App
Web-Based Address Book	Car Payment Calculator App	Flag Quiz App
Enhanced Painter App	Mileage Calculator App	Phone Book App with Data Binding
PhotoViewer App	Body Mass Index Calculator App	Enhanced UsingGradients App
Data Bindings to LINQ queries	Target-Heart-Rate Calculator App	Enhanced DrawStars app
Snake PolyLine App	Phone-Book Web Service	Image Reflector App
Drawing App		Accessibility: Speech-Controlled Drawing App
Enhanced Tip Calculator App		
Mortgage Calculator App		

Fig. 4 | A sampling of the online chapters' exercises.

Web App Development with ASP.NET
Microsoft's .NET server-side technology, ASP.NET, enables you to create robust, scalable web-based apps. You'll build several apps, including a web-based guestbook that uses ASP.NET and the ADO .NET Entity Framework to store data in a database and display data in a web page.

Extensible Markup Language (XML)
The Extensible Markup Language (XML) is pervasive in the software-development industry, e-business and throughout the .NET platform. It's used in most of this book's online topics. XML is required to understand XAML—a Microsoft XML vocabulary that's used to describe graphical user interfaces, graphics and multimedia for Universal Windows Platform (UWP) GUI, graphics and multimedia apps, Windows 10 Mobile apps and Windows Presentation Foundation (WPF) apps. We present XML fundamentals, then discuss LINQ to XML, which allows you to query XML content using LINQ syntax.

Universal Windows Platform (UWP) for Desktop and Mobile Apps
The Universal Windows Platform (UWP) is designed to provide a common platform and user experience across all Windows devices, including personal computers, smartphones,

tablets, Xbox and even Microsoft's new HoloLens virtual reality and augmented reality holographic headset—all using nearly identical code. We present GUI, graphics and multimedia apps, and demonstrate them on both personal computers and the smartphone emulator that comes with Visual Studio 2015 Community edition.

REST Web Services
Web services enable you to package app functionality in a manner that turns the web into a library of *reusable* services. We include a case study on building a math question generator web service that's called by a math tutor app.

Building a Microsoft Azure™ Cloud Computing App
Microsoft Azure's web services enable you to develop, manage and distribute your apps in "the cloud." We'll demonstrate how to use Azure web services to store an app's data online.

Windows Presentation Foundation (WPF) GUI, Graphics and Multimedia
Windows Presentation Foundation (WPF)—created after Windows `Forms` and before UWP—is another Microsoft technology for building robust GUI, graphics and multimedia desktop apps. WPF provides you with complete control over all aspects of a GUI's look-and-feel and includes multimedia capabilities that are not available in Windows `Forms`. We discuss WPF in the context of a painting app, a text editor, a color chooser, a book-cover viewer, a television video player, various animations, and speech synthesis and recognition apps.

We're moving away from WPF in favor of UWP for creating apps that can run on desktop, mobile and other Windows devices. For this reason, the WPF introduction is provided *as is* from the previous edition—we will no longer evolve this material.

Optional Case Study: Using the UML to Develop an Object-Oriented Design and C# Implementation of an ATM (Automated Teller Machine)
The UML™ (Unified Modeling Language™) is the industry-standard graphical language for visually modeling object-oriented systems. We introduce the UML in the early chapters and provide an optional online object-oriented design case study in which we use the UML to design and implement the software for a simple ATM. We analyze a typical *requirements document* that specifies the details of the system to be built. We determine the *classes* needed to implement that system, the *attributes* the classes need to have, the *behaviors* the classes' methods need to exhibit and we specify how the classes must *interact* with one another to meet the system requirements. From the design, we produce a complete working C# implementation. Students often report a "light bulb moment"—the case study helps them "tie it all together" and truly understand object orientation.

Teaching Approach

Visual C# How to Program, 6/e contains a rich collection of examples. We concentrate on building well-engineered software and stress program clarity.

Live-Code Approach. The book is loaded with "live-code" examples—most new concepts are presented in the context of complete working Visual C# apps, followed by one or more executions showing program inputs and outputs. In the few cases where we show a code snippet, to ensure correctness first we tested it in a complete working program then copied the code from the program and pasted it into the book.

Syntax Shading. For readability, we syntax shade the code, similar to the way Visual Studio colors the code. Our syntax-shading conventions are:

```
comments appear like this
keywords appear like this
constants and literal values appear like this
all other code appears in black
```

Code Highlighting. We emphasize key code segments by placing them in gray rectangles.

Using Fonts for Emphasis. We place the key terms and the index's page reference for each defining occurrence in colored **bold** text for easy reference. We show on-screen components in the **bold Helvetica** font (for example, the **File** menu) and Visual C# program text in the `Lucida` font (for example, `int count = 5;`). We use *italics* for emphasis.

Objectives. The chapter objectives preview the topics covered in the chapter.

Programming Tips. We include programming tips to help you focus on important aspects of program development. These tips and practices represent the best we've gleaned from a combined nine decades of programming and teaching experience.



Good Programming Practices

The Good Programming Practices *call attention to techniques that will help you produce programs that are clearer, more understandable and more maintainable.*



Common Programming Errors

Pointing out these Common Programming Errors *reduces the likelihood that you'll make them.*



Error-Prevention Tips

These tips contain suggestions for exposing and removing bugs from your programs; many of the tips describe aspects of Visual C# that prevent bugs from getting into programs.



Performance Tips

These tips highlight opportunities for making your programs run faster or minimizing the amount of memory that they occupy.



Portability Tips

These tips help you write code that will run on a variety of platforms.



Software Engineering Observations

The Software Engineering Observations *highlight architectural and design issues that affect the construction of software systems, especially large-scale systems.*



Look-and-Feel Observation 3.1

These observations help you design attractive, user-friendly graphical user interfaces that conform to industry norms.

Summary Bullets. We present a detailed bullet-list summary of each chapter.

Terminology. We include an alphabetized list of the important terms defined in each chapter.

Index. We've included an extensive index for reference. Defining occurrences of key terms in the index are highlighted with a colored **bold** page number.

Obtaining the Software Used in *Visual C# How to Program, 6/e*

We wrote the code examples in *Visual C# How to Program, 6/e* using Microsoft's free Visual Studio 2015 Community edition. See the Before You Begin section that follows this preface for download and installation instructions.

Instructor Supplements

The following supplements are available to ***qualified instructors only*** through Pearson Education's online Instructor Resource Center at www.pearsonglobaleditions.com/deitel:

- ***Solutions Manual*** contains solutions to *most* of the end-of-chapter exercises. We've included many *Making-a-Difference* exercises, most with solutions. **Please do not write to us requesting access to the Pearson Instructor's Resource Center. Access is restricted to college instructors who have adopted the book for their courses. Instructors can obtain access through their Pearson representatives.** If you're not a registered faculty member, contact your Pearson representative or visit `http://www.pearsonglobaleditions.com/deitel`. Exercise Solutions are *not* provided for "project" exercises. Check out our Programming Projects Resource Center for lots of additional exercise and project possibilities:

```
http://www.deitel.com/ProgrammingProjects
```

- ***Test Item File*** of multiple-choice questions (approximately two per top-level book section)
- ***Customizable PowerPoint® slides*** containing all the code and figures in the text, plus bulleted items that summarize the key points in the text.

Microsoft DreamSpark™

Microsoft provides many of its professional developer tools to students for free via a program called DreamSpark (`http://www.dreamspark.com`). See the website for details on verifying your student status so you take advantage of this program. To compile, test, debug and run this book's examples, you need only Windows 10 and the free Visual Studio 2015 Community edition. With the exception of the online UWP examples, the book's examples also will compile and run on Windows 7 and higher.

Acknowledgments

We'd like to thank Barbara Deitel of Deitel & Associates, Inc. She painstakingly researched the new capabilities of Visual C#, Visual Studio, .NET and other key technologies. We'd also like to acknowledge Frank McCown, Ph.D., Associate Professor of Computer Science, Harding University for his suggestion to include an example that used a `ProgressBar` with `async` and `await` in Chapter 23—so we ported to C# a similar example from our textbook *Java How to Program, 10/e*.

We're fortunate to have worked with the dedicated team of publishing professionals at Pearson Higher Education. We appreciate the guidance, wisdom, energy and mentorship of Tracy Johnson, Executive Editor, Computer Science. Kristy Alaura did an extraordinary job recruiting the book's reviewers and managing the review process. Bob Engelhardt did a wonderful job bringing the book to publication.

Reviewers

The book was scrutinized by academics teaching C# courses and industry C# experts. They provided countless suggestions for improving the presentation. Any remaining flaws in the book are our own.

Sixth Edition Reviewers: Qian Chen (Department of Engineering Technology: Computer Science Technology Program, Savannah State University), Octavio Hernandez (Microsoft Certified Solutions Developer, Principal Software Engineer at Advanced Bionics), José Antonio González Seco (Parliament of Andalusia, Spain), Bradley Sward (College of Dupage) and Lucian Wischik (Microsoft Visual C# Team).

Fifth Edition Post-Publication Reviewers: To help us prepare to write 6/e, the following academics reviewed 5/e and provided many helpful suggestions: Qian Chen (Savannah State University), Hongmei Chi (Florida A&M University), Kui Du (University of Massachusetts, Boston), James Leasure (Cuyahoga Community College West), Victor Miller (Ramapo College), Gary Savard (Champlain College) and Mohammad Yusuf (New Hampshire Technical Institute).

Other recent edition reviewers: Douglas B. Bock (MCSD.NET, Southern Illinois University Edwardsville), Dan Crevier (Microsoft), Shay Friedman (Microsoft Visual C# MVP), Amit K. Ghosh (University of Texas at El Paso), Marcelo Guerra Hahn (Microsoft), Kim Hamilton (Software Design Engineer at Microsoft and co-author of *Learning UML 2.0*), Huanhui Hu (Microsoft Corporation), Stephen Hustedde (South Mountain College), James Edward Keysor (Florida Institute of Technology), Narges Kasiri (Oklahoma State University), Helena Kotas (Microsoft), Charles Liu (University of Texas at San Antonio), Chris Lovett (Software Architect at Microsoft), Bashar Lulu (INETA Country Leader, Arabian Gulf), John McIlhinney (Spatial Intelligence; Microsoft MVP Visual Developer, Visual Basic), Ged Mead (Microsoft Visual Basic MVP, DevCity.net), Anand Mukundan (Architect, Polaris Software Lab Ltd.), Dr. Hamid R. Nemati (The University of North Carolina at Greensboro), Timothy Ng (Microsoft), Akira Onishi (Microsoft), Jeffrey P. Scott (Blackhawk Technical College), Joe Stagner (Senior Program Manager, Developer Tools & Platforms, Microsoft), Erick Thompson (Microsoft), Jesús Ubaldo Quevedo-Torrero (University of Wisconsin–Parkside, Department of Computer Science), Shawn Weisfeld (Microsoft MVP and President and Founder of UserGroup.tv) and Zijiang Yang (Western Michigan University).

As you read the book, we'd sincerely appreciate your comments, criticisms, corrections and suggestions for improving the text. Please address all correspondence to:

```
deitel@deitel.com
```

We'll respond promptly. It was fun writing *Visual C# How to Program, 6/e*—we hope you enjoy reading it!

Paul Deitel
Harvey Deitel

About the Authors

Paul Deitel, CEO and Chief Technical Officer of Deitel & Associates, Inc., has over 35 years of experience in computing. He is a graduate of MIT, where he studied Information Technology. Through Deitel & Associates, Inc., he has delivered hundreds of programming courses worldwide to clients, including Cisco, IBM, Boeing, Siemens, Sun Microsystems (now Oracle), Dell, Fidelity, NASA at the Kennedy Space Center, the National Severe Storm Laboratory, NOAA (National Oceanic and Atmospheric Administration), White Sands Missile Range, Rogue Wave Software, SunGard, Nortel Networks, Puma, iRobot, Invensys and many more. He and his co-author, Dr. Harvey Deitel, are the world's best-selling programming-language textbook/professional book/video authors.

Paul was named as a Microsoft® Most Valuable Professional (MVP) for C# in 2012–2014. According to Microsoft, "the Microsoft MVP Award is an annual award that recognizes exceptional technology community leaders worldwide who actively share their high quality, real-world expertise with users and Microsoft." He also holds the Java Certified Programmer and Java Certified Developer designations and is an Oracle Java Champion.



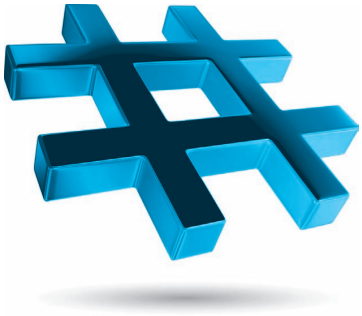
C# MVP 2012–2014

Dr. Harvey Deitel, Chairman and Chief Strategy Officer of Deitel & Associates, Inc., has over 55 years of experience in the computer field. Dr. Deitel earned B.S. and M.S. degrees in Electrical Engineering from MIT and a Ph.D. in Mathematics from Boston University—he studied computing in each of these programs before they spun off Computer Science programs. He has extensive college teaching experience, including earning tenure and serving as the Chairman of the Computer Science Department at Boston College before founding Deitel & Associates, Inc., in 1991 with his son, Paul. The Deitels' publications have earned international recognition, with translations published in Japanese, German, Russian, Spanish, French, Polish, Italian, Simplified Chinese, Traditional Chinese, Korean, Portuguese, Greek, Urdu and Turkish. Dr. Deitel has delivered hundreds of programming courses to academic, corporate, government and military clients.

Acknowledgments for the Global Edition

Pearson would like to thank and acknowledge Komal Arora for contributing to the Global Edition, and Ela Kashyap, Amity University Noida, Siddharth Nair, and Sandeep Singh, Jaypee Institute of Information Technology for reviewing the Global Edition.

This page intentionally left blank



Before You Begin

Please read this section before using the book to ensure that your computer is set up properly.

Font and Naming Conventions

We use fonts to distinguish between features, such as menu names, menu items, and other elements that appear in the program-development environment. Our convention is

- to emphasize Visual Studio features in a **sans-serif bold font** (e.g., **Properties** window) and
- to emphasize program text in a fixed-width `sans-serif font` (e.g., `bool x = true`).

Visual Studio 2015 Community Edition

This textbook uses Windows 10 and the free Microsoft Visual Studio 2015 Community edition—Visual Studio also can run on various older Windows versions. Ensure that your system meets Visual Studio 2015 Community edition's minimum hardware and software requirements listed at:

```
https://www.visualstudio.com/en-us/visual-studio-2015-system-
   requirements-vs
```

Next, download the installer from

```
https://www.visualstudio.com/products/visual-studio-express-vs
```

then execute it and follow the on-screen instructions to install Visual Studio.

Though we developed the book's examples on Windows 10, the examples will run on Windows 7 and higher—with the exception of those in our online Universal Windows Platform (UWP) presentation. Most examples without graphical user interfaces (GUIs) also will run on other C# and .NET implementations—see "If You're Not Using Microsoft Visual C#…" later in this Before You Begin for more information.

Viewing File Extensions

Several screenshots in *Visual C# How to Program, 6/e* display file names with file-name extensions (e.g., `.txt`, `.cs`, `.png`, etc.). You may need to adjust your system's settings to display file-name extensions. If you're using Windows 7:

1. Open **Windows Explorer**.
2. Press the *Alt* key to display the menu bar, then select **Folder Options...** from the **Tools** menu.
3. In the dialog that appears, select the **View** tab.

4. In the **Advanced settings** pane, *uncheck* the box to the left of the text **Hide extensions for known file types**.
5. Click **OK** to apply the setting and close the dialog.

If you're using Windows 8 or higher:

1. Open **File Explorer**.
2. Click the **View** tab.
3. Ensure that the **File name extensions** checkbox is *checked*.

Obtaining the Source Code

Visual C# How to Program, 6/e's source-code examples are available for download at

```
http://www.pearsonglobaleditions.com/deitel
```

Click the **Examples** link to download the ZIP archive file to your computer—most browsers will save the file into your user account's `Downloads` folder. You can extract the ZIP file's contents using built-in Windows capabilities, or using a third-party archive-file tool such as WinZip (`www.winzip.com`) or 7-zip (`www.7-zip.org`).

Throughout the book, steps that require you to access our example code on your computer assume that you've extracted the examples from the ZIP file and placed them in your user account's `Documents` folder. You can extract them anywhere you like, but if you choose a different location, you'll need to update our steps accordingly. To extract the ZIP file's contents using the built-in Windows capabilities:

1. Open **Windows Explorer** (Windows 7) or **File Explorer** (Windows 8 and higher).
2. Locate the ZIP file on your system, typically in your user account's `Downloads` folder.
3. Right click the ZIP file and select **Extract All...**.
4. In the dialog that appears, navigate to the folder where you'd like to extract the contents, then click the **Extract** button.

Configuring Visual Studio for Use with This Book

In this section, you'll use Visual Studio's **Options** dialog to configure several Visual Studio options. Setting these options is not required, but will make your Visual Studio match what we show in the book's Visual Studio screen captures.

Visual Studio Theme

Visual Studio has three color themes—**Blue**, **Dark** and **Light**. We used the **Blue** theme with light colored backgrounds to make the book's screen captures easier to read. To switch themes:

1. In the Visual Studio **Tools** menu, select **Options...** to display the **Options** dialog.
2. In the left column, select **Environment**.
3. Select the **Color theme** you wish to use.

Keep the **Options** dialog open for the next step.

Line Numbers

Throughout the book's discussions, we refer to code in our examples by line number. Many programmers find it helpful to display line numbers in Visual Studio as well. To do so:

1. Expand the **Text Editor** node in the **Options** dialog's left pane.
2. Select **All Languages**.
3. In the right pane, check the **Line numbers** checkbox.

Keep the **Options** dialog open for the next step.

Tab Size for Code Indents

Microsoft recommends four-space indents in source code, which is the Visual Studio default. Due to the fixed and limited width of code lines in print, we use three-space indents—this reduces the number of code lines that wrap to a new line, making the code a bit easier to read. If you wish to use three-space indents:

1. Expand the **C#** node in the **Options** dialog's left pane and select **Tabs**.
2. Ensure that **Insert spaces** is selected.
3. Enter **3** for both the **Tab size** and **Indent size** fields.
4. Click **OK** to save your settings.

If You're Not Using Microsoft Visual C#...

C# can be used on other platforms via two open-source projects managed by the .NET Foundation (`http://www.dotnetfoundation.org`)—the Mono Project and .NET Core.

Mono Project

The Mono Project is an open source, cross-platform C# and .NET Framework implementation that can be installed on Linux, OS X (soon to be renamed as macOS) and Windows. The code for most of the book's console (non-GUI) apps will compile and run using the Mono Project. Mono also supports Windows `Forms` GUI, which is used in Chapters 14–15 and several later examples. For more information and to download Mono, visit:

```
http://www.mono-project.com/
```

.NET Core

.NET Core is a new cross-platform .NET implementation for Windows, Linux, OS X and FreeBSD. The code for most of the book's console (non-GUI) apps will compile and run using .NET Core. At the time of this writing, a .NET Core version for Windows was available and versions were still under development for other platforms. For more information and to download .NET Core, visit:

```
https://dotnet.github.io/
```

You're now ready to learn C# and the .NET platform with *Visual C# How to Program, 6/e*. We hope you enjoy the book!